



Free-Slip boundary conditions on curved boundaries

Louis Moresi¹ 

¹Australian National University

Keywords Underworld Code, Tricks of the Trade, development

Free-slip boundary conditions are used to simplify the physical behaviour at a domain boundary. It may be a free-surface where the boundary deforms slightly in response to the internal flow, or it may be an interface where the boundary layer thickness is so small that it cannot be resolved at the same time as the interior flow. The simplifying assumption: ignore the changes in shape, ignore the thin boundary layer, treat the surface as impenetrable, and the tangential stresses as vanishingly small.

$$\mathbf{u} \cdot \hat{\mathbf{n}} = 0 \quad \text{and} \quad \hat{\mathbf{t}} \cdot \boldsymbol{\sigma} \cdot \hat{\mathbf{n}} = 0. \quad (1)$$

In the weak form, boundary tractions appear as surface integrals. Multiplying the momentum balance by a test function $\mathbf{w}$ and integrating by parts gives

$$\begin{aligned} & \int_{\Omega} \boldsymbol{\sigma} : \nabla \mathbf{w} ; \mathrm{d}V \\ & - \int_{\partial\Omega} (\boldsymbol{\sigma} \cdot \hat{\mathbf{n}}) \cdot \mathbf{w} ; \mathrm{d}S = \\ & \int_{\Omega} \mathbf{f} \cdot \mathbf{w} ; \mathrm{d}V . \end{aligned}$$

\$\$

Drop the surface integral and you have imposed zero traction in all directions — free *everything* (a free surface), not free slip. Constrain the surface-normal degrees of freedom and the surface integral only addresses the tangential traction terms.

On a Cartesian box, the first expression in (1) constrains a single velocity component. If you hold u_x fixed on a vertical wall, the solver removes a row of unknowns, and there is nothing further to discuss. On a sphere, an annulus, a mesh that does not align with the coordinates, or a surface with topography, $\mathbf{u} \cdot \hat{\mathbf{n}}$ is not a single component of the unknown — it constrains a combination of unknowns at each point, and leaves other combinations free. That has the potential to make a simplifying assumption complicated to implement.

Let's assume, for a moment, we confine ourselves to simple domains such as an annulus, or a spherical shell, which are commonly used for planetary modelling. For each of these cases, there are coordinate systems, and well known forms of the differential operators that do restore the boundary condition to being a constraint in a single direction. Admittedly this requires reformulating all the equations, but for a symbolic-first code such as underworld, this is possible. This is the strategy used by CITCOMS [Zhong et al]. But not every domain

boundary has a convenient coordinate system to follow. Even accounting for slight ellipticity introduces significant complexity in all the differential operators; anything more complicated will not have any useful coordinate reformulation.

The second condition is also worth noting. We don't think about this when we constrain a degree of freedom, the other one/s, left unconstrained are *natural* to the problem. They fall out as traction-free surface conditions automatically in a finite element weak form. If we cannot simply eliminate one degree of freedom at each point, how **do** we satisfy all the parts of (1) ?

1. FOUR POSSIBILITIES

We outline four possible approaches (four you can try out in Underworld3). They fall into two pairs: two impose the constraint **weakly**, by adding a term to the momentum equation and letting the solution satisfy the condition to within the accuracy of that term: a direct penalty, and Nitsche's method, which differ in whether the term is consistent. Two impose it **exactly**: by construction, changing the basis so that the constraint is a component that can be struck out, or by a Lagrange multiplier, adding an equation that enforces it. The weak pair have a parameter to select that may need to be tuned for each problem and a floor they cannot go below. The exact pair are not tuneable, and they both return the boundary traction as a side-effect of the solution.

1.a. 1. A direct penalty

This could not be more simple, conceptually. We are working in a variational environment, so we just add into our equation system, a term that punishes any flow through the boundary:

$$\dots + \kappa \int_{\partial\Omega} (\mathbf{u} \cdot \hat{\mathbf{n}})(\mathbf{w} \cdot \hat{\mathbf{n}}) \, \mathrm{d}S.$$

\$\$

κ is a bare number, with none of the μ/h that Nitsche's parameter carries below. It has to absorb the scale of the problem itself, which is why the value that works is a property of the model rather than a default.

One line, no new machinery, and it works on any geometry. It is still in a good many working scripts, and deservedly. What you are solving is a perturbed problem, though, and it is perturbed by exactly the amount the constraint is violated: the discrete solution sits where the penalty term balances the traction it is fighting and this leaves $\mathbf{u} \cdot \hat{\mathbf{n}}$ small but not zero.

Making it smaller means pushing harder, and pushing harder degrades the condition-number of the operator. The error is traded against the conditioning, and (see the table below), this trade-off eventually stops returning any benefit: past 10^4 the boundary constraint stops improving (the surface is leaky), and by 10^6 the solve fails.

Underworld spells it as a boundary traction opposing normal flow, using the surface normal at the quadrature points (Γ) provided by PETSc:

```
G = mesh.Gamma
penalty = 10000
stokes.add_natural_bc(penalty * G.dot(v.sym) * G, "Upper")
```

1.b. Nitsche's method

The reason the penalty is only accurate in the limit is that it is not *consistent*: substituting the true solution does not leave the equation satisfied, because the true solution is subject to a boundary traction the penalty form ignores. Nitsche's method (Nitsche, 1971) restores consistency by carrying that traction explicitly:

\$\$ \dots

- $\int_{\partial\Omega} (\hat{\mathbf{n}} \cdot \boldsymbol{\sigma}(\mathbf{u}) \cdot \hat{\mathbf{n}}) (\mathbf{w} \cdot \hat{\mathbf{n}}) ; \mathrm{d}S$
- $\int_{\partial\Omega} (\hat{\mathbf{n}} \cdot \boldsymbol{\sigma}(\mathbf{w}) \cdot \hat{\mathbf{n}}) (\mathbf{u} \cdot \hat{\mathbf{n}}) ; \mathrm{d}S$
- $\frac{\gamma}{2} \int_{\partial\Omega} (\mathbf{u} \cdot \hat{\mathbf{n}}) (\mathbf{w} \cdot \hat{\mathbf{n}}) ; \mathrm{d}S$.

\$\$

The first of the three is the consistency term: it is the boundary traction the integration by parts produced, and including this makes the true solution satisfy the discrete equations exactly. The second is its transpose, which keeps the form symmetric and buys optimal convergence in L^2 . The third is the penalty again, and it is still needed — but now for *stability* rather than for accuracy, and γ has a threshold set by an inverse inequality rather than being a dial you turn up until the answer looks right.

This is a real improvement and it is still done through a weak imposition. The constraint holds to the accuracy of the discretisation, not to the accuracy of the arithmetic — measured below, it leaks a few parts in a thousand on a typical mesh, and the leak falls with increasing mesh resolution.

1.c. *A constraint equation, with a multiplier*

The two above add a *term* to the weak form of the equation. This one adds an *equation*.

Carry a scalar field λ on the boundary and require, as a row of the system in its own right,

$$\int_{\partial\Omega} (\mathbf{u} \cdot \hat{\mathbf{n}}) - \tilde{u}_n, q ; \mathrm{d}S = 0 \quad \text{for all } q,$$

\$\$

where $\tilde{u}_n$ is the prescribed wall-normal velocity — zero for free slip, and a datum if the wall is being driven — and λ enters the momentum row as the traction $\lambda \hat{\mathbf{n}}$ that holds the constraint. It is a Lagrange multiplier, and the system becomes a larger saddle point: velocity, pressure, and now λ .

λ has units of stress. At convergence it is σ_{nn} on that boundary, so dividing by $\Delta\rho g$ — density contrast times gravity, no relation to the constraint datum — turns it into a dynamic topography.

The constraint row is exact, so unlike a penalty there is no parameter whose size decides how well it holds. Two practical things do have to be dealt with.

- λ is carried as a full-domain field but only its boundary trace means anything, so the interior degrees of freedom are constrained out of the global system in the section before the solve sees it. They are not solved for and they do not enlarge the $[p, \lambda]$ block.

- The $[p, \lambda]$ Schur complement is poorly conditioned on its own, so an augmented-Lagrangian term $r(\mathbf{u} \cdot \hat{\mathbf{n}} - \tilde{u}_n)\hat{\mathbf{n}}$ is added to the momentum row. It does not change what the constraint enforces, because the λ row still carries the exact constraint.

```
stokes = uw.systems.Stokes_Constrained(mesh, velocityField=v, pressureField=p)
lam = stokes.add_constraint_bc(0.0, "Upper")
stokes.solve()
```

And the reason to care about it beyond the constraint: **at convergence the momentum row's boundary term is the normal traction**. It is not recovered from the velocity field afterwards — it is an unknown the solve returned, available through `traction` and, divided by $\Delta\rho g$, through `topography`.

That term is the whole boundary load, $\lambda + r(\mathbf{u} \cdot \hat{\mathbf{n}} - \tilde{u}_n)$, and not the multiplier alone. The second part vanishes only where the constraint row is satisfied exactly; discretely it is satisfied to the solver's tolerance, and r multiplies that residual back into the traction. With a viscosity-weighted r and a lateral viscosity contrast it is most of the answer, so the two parts are not separable in practice.

1.d. Rotating the degrees of freedom

In this approach, we stop “asking for” the constraint and just impose it. At each constrained node, we change the coordinate basis in which the velocity unknowns are expressed, from the global Cartesian frame to the local $(\hat{\mathbf{n}}, \hat{\mathbf{t}})$ frame. In that basis “no flow through the boundary” is again a single component, and it is removed the same way it would be on a box.

Collect the per-node rotations into a block-diagonal Q , equal to the identity at every node that is not constrained. The rotated system is

$$\hat{\mathbf{A}} = Q^T \mathbf{A} Q, \quad \hat{\mathbf{b}} = Q^T \mathbf{b}, \quad \hat{\mathbf{u}} = Q \mathbf{u},$$

and the wall-normal row of $\hat{\mathbf{A}}$ is struck out. The constraint then holds to machine precision, because it is not being solved for at all.

This is the classical strategy. It is in the early finite-element literature, and Engelman et al. (1982) were already reviewing the alternatives and choosing between them on grounds of global mass conservation in 1982. What is worth explaining is not the idea but why, given that it is exact and the others are not, it is the least used of the three.

2. TRADE-OFFS

Rotating the degrees of freedom leaves the discrete problem in a **mixed basis**. Interior nodes hold (u_x, u_y) ; constrained nodes hold (u_n, u_t) . Nothing about that is difficult in itself, but everything downstream has to agree about which nodes are which.

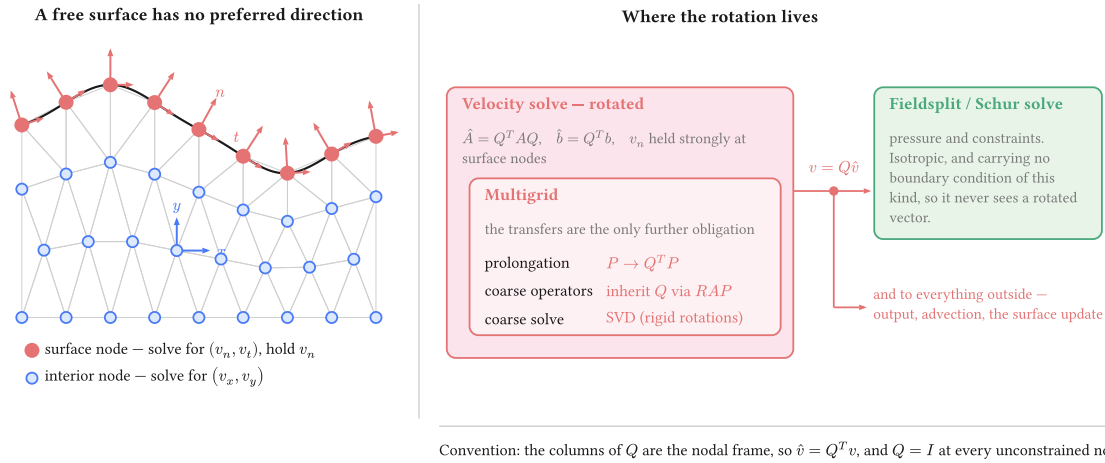


Figure 1: Where the rotation lives. The obligation is contained: the velocity solve is rotated and carries its multigrid with it, while the Schur complement and the pressure solve beside it never handle a rotated vector, because the pressure block carries no boundary condition of this kind. One un-rotation sits on the boundary between them and feeds both.

Four things carry Q : the operator, the right-hand side, the solution on the way out, and the multigrid prolongation. The coarse operators inherit it through the Galerkin triple product rather than being rotated separately, and the coarse solve should be an SVD, because a Galerkin-coarsened rotated operator inherits any rigid-rotation null space of the constrained problem (the exact constraint makes the null space of the sphere and the annulus a dominant feature of the solve).

We do not know the cost of the additional complexity on the solver and setup times, or on the accuracy of the solution but this can be measured and will differ from problem to problem.

3. CHOICE OF THE SURFACE NORMAL

A question with a less obvious answer than it looks, and the measurements further down say it is the most consequential choice in the note. The boundary of a discretised domain is a set of straight facets, and the assembled constraint is an integral over those facets. The node normal consistent with that integral is the average of the adjacent facet normals **weighted by facet measure** — not the normal of the smooth surface the mesh approximates, and not the facet normal on its own.

This is the consistent normal of Engelman et al. (1982), and we should say so plainly: we re-derived it from the assembled boundary integral, and Engelman, Sani and Gresho derived it in 1982 from global conservation of mass, which is the same object arrived at from the other side. Their paper is about exactly this — how to impose a normal or tangential condition on a boundary that does not line up with the coordinate directions — and the recommendation is the one here.

Using the analytic normal is exact for the geometry and therefore inconsistent with the discretisation, which is the wrong way round: the solver is not solving on the sphere, it is solving on the polyhedron. In parallel the same argument has a sharper edge, because a normal accumulated rank-locally is wrong at a partition boundary, where a node's facets are split across ranks and no rank sees them all.

Using the **facet** normal is worse than inconsistent, and this is the one place in the note where a choice does real damage. Imposing $u \cdot \hat{n} = 0$ facet by facet asks a node shared by two facets to satisfy two different constraints,

and two independent constraints on a two-component velocity leave nothing. Push the coefficient up and the vertex velocities go to zero: the flow is being asked to stay inside a polygon rather than a circle, and the discrete limit is a different problem from the smooth one. Refining the mesh does not approach the smooth answer, because it is not converging to it.

A direct penalty at $\kappa = 10^6$, on the annulus of the next section, against each normal in turn:

cell size	facet normal: leak / velocity error / stress error	node normal: leak / velocity / stress
0.150	8.9×10^{-6} / 0.60 / 0.21	4.6×10^{-5} / 1.0×10^{-2} / 2.4×10^{-2}
0.075	1.1×10^{-5} / 0.61 / 0.25	3.7×10^{-5} / 2.4×10^{-3} / 6.3×10^{-3}
0.050	1.9×10^{-5} / 0.60 / 0.26	3.1×10^{-5} / 1.0×10^{-3} / 2.7×10^{-3}
0.035	2.3×10^{-5} / 0.59 / 0.26	3.1×10^{-5} / 4.9×10^{-4} / 1.4×10^{-3}

The facet-normal column is stuck: the velocity is 60% wrong and the stress 26% wrong, and refining changes neither. Note what the leak does — it is *excellent*, better than the node normal’s, because a frozen boundary passes no flow. A metric that only asks whether the constraint is satisfied cannot tell a locked solution from a good one.

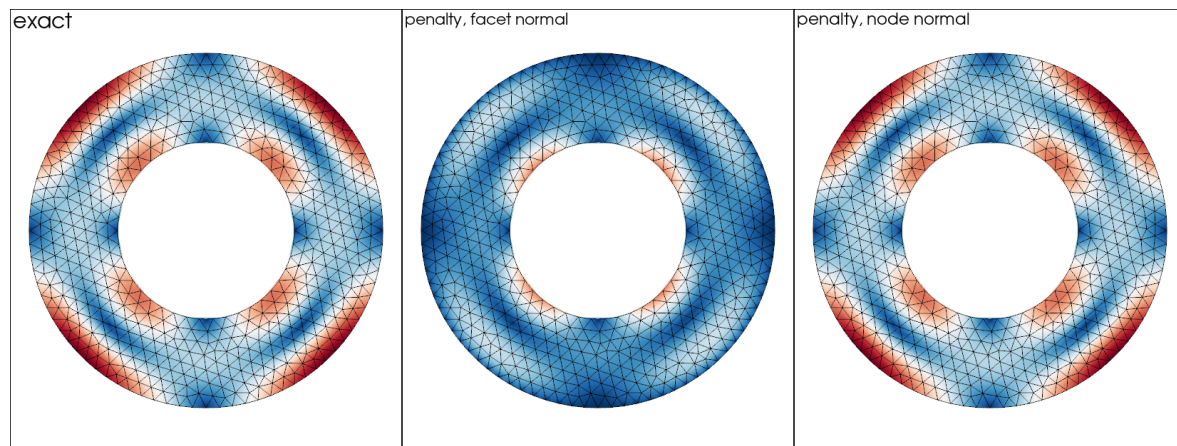


Figure 2: The same problem, the same coefficient, the same colour scale. Against the facet normal the flow along the outer boundary is suppressed — the peak speed falls by a quarter and the two fast lobes at the boundary are gone. Against the measure-weighted node normal it is the exact solution.

Everything that follows uses the node normal, which is what `add_nitsche_bc` and `add_rotated_freeslip_bc` take by default and what `mesh.boundary_normal` returns. The facet normal does not appear again.

The consistent normal is not the end of the matter. Behr (2004) takes it as the starting point — “preferred from the point of view of conservation” — and reports that in sloshing problems it still does not guarantee a good discrete slip condition, with non-physical recirculation appearing at curved walls; the remedies offered there are the Navier slip condition and a “BC-free” boundary. We have read the abstract rather than the paper, and have not looked for that recirculation in our own cases. It is the obvious thing to test next for anyone running free slip on a strongly curved wall.

4. WHEN THE CHOICE MATTERS

Solve a convection model with any of these approaches, and the velocity field is the same to plotting accuracy, as long as each is set up correctly. Generally speaking, a leak of order 10^{-3} or 10^{-4} in $\mathbf{u} \cdot \hat{\mathbf{n}}$ is within the expected accuracy of the solution on the mesh and the main driver of which method to choose should be solver efficiency (wall time).

The difference in the methods appears when the wall-normal traction is a required output of the model: dynamic topography, geoid, gravity, or a plate-boundary force balance require accurate integration of boundary stresses. Here the choice becomes more subtle, and the methods have quite different accuracies, and different efficiencies.

4.a. The surface permeability question

An annulus, no slip on the inner radius, the treatment under test on the outer, driven by a degree-four radial density anomaly. The number is the largest normal velocity on the outer boundary, taken against the true radial direction and divided by the flow speed: the fraction of the flow going through a boundary nothing should pass through. Penalty at 10^4 , Nitsche at $\gamma = 10$. [REFERENCE TO ANALYTIC MODULE]

cell size	penalty	Nitsche	multiplier	rotated
0.150	3.3×10^{-3}	4.6×10^{-3}	8.3×10^{-4}	7.3×10^{-11}
0.100	2.7×10^{-3}	2.2×10^{-3}	1.6×10^{-4}	6.5×10^{-11}
0.075	2.6×10^{-3}	1.7×10^{-3}	5.6×10^{-5}	1.0×10^{-10}
0.050	2.6×10^{-3}	5.8×10^{-4}	1.2×10^{-5}	8.4×10^{-11}

Nitsche leaks parts in a thousand and improves roughly as $h^{1.9}$ — the rate consistency buys. The multiplier starts an order of magnitude better and falls much faster, near $h^{3.9}$. The rotated constraint does not move at all: it sits at the solver's floor at every resolution, because the mesh has nothing to do with it. The penalty does not improve with the mesh either, and for the opposite reason: its leak is set by the coefficient, not by the discretisation.

4.b. What each parameter buys

The two weak methods look alike in that table. They are not alike, and their own parameters are what tells them apart.

κ (penalty)	leak	γ (Nitsche)	leak
10^2	2.6×10^{-1}	1	diverged
10^3	2.6×10^{-2}	10	1.7×10^{-3}
10^4	2.6×10^{-3}	100	2.7×10^{-4}
10^5	3.0×10^{-4}	1000	3.0×10^{-5}
10^6	4.5×10^{-5}	10^4 and above	diverged

Nitsche is bounded at both ends. Below $\gamma \sim 1$ the form is no longer coercive and no amount of solver tuning recovers it; from $\gamma \sim 10^4$ in this problem, the line search stops converging. The virtue of $\gamma \sim 10$ is that it sits within that window on any mesh, because γ is dimensionless and the term it scales already carries μ/h .

The penalty coefficient scales differently because it directly penalises the value of the velocity across the boundary. It should therefore scale with the characteristic velocity which is best estimated from the magnitude of the forcing terms and the resisting viscosity.

Written against the node normal, the penalty simply trades: a decade of coefficient for a decade of leak, all the way to 10^6 , with no wall in this problem. What it does not do is converge — the leak is bought with the parameter rather than with the mesh — so the coefficient has to be re-chosen whenever the forcing or the viscosity changes.

4.c. The stress, measured

The leak says how well each treatment satisfied the free-slip requirement at the boundary. But we still need to measure whether the method converges to the correct solution.

Kramer, Davies and Wilson (Kramer et al., 2021) give exact Stokes solutions in a cylindrical annulus, and their `assess` package also publishes the radial stress. Underworld wraps it as `uw.analytic.CylindricalStokes`. The case here is the smooth one: a density anomaly $(r/r_o)^k \cos n\theta$ with $n = 2$ and $k = 3$, viscosity 1, free slip on both radii. On the outer boundary the exact radial stress is a single harmonic,

$$\sigma_{rr}(r_o, \theta) = 0.1506696 \cos 2\theta,$$

\$\$

fitted to a residual of 10^{-16} , so the whole of the surface stress is that one amplitude and the metric is its relative error. The inner boundary carries the exact analytic velocity as a Dirichlet condition instead of a free-slip treatment of its own.

There are two routes to the surface stress and the difference between them is the point of the section:

- **recovered** — project $\hat{n} \cdot \sigma \cdot \hat{n}$ out of the solved velocity and pressure. Every approach offers this, but it is the only route the weak ones have.
- **reaction** — compute the boundary-normal traction for the rotated constraint, and use the multiplier field for the constraint method. This is not recovered from the stress field but from the surface reaction to the stress field.

Both are computed against the analytic radial direction, which is also the direction the oracle publishes, so no treatment is scored against its own normal.

cell size	penalty	Nitsche	multiplier	rotated	rotated, reaction	multiplier, reaction
0.150	2.5×10^{-2}	5.9×10^{-2}	2.4×10^{-2}	2.4×10^{-2}	6.8×10^{-3}	8.6×10^{-3}
0.100	1.1×10^{-2}	2.4×10^{-2}	1.0×10^{-2}	1.0×10^{-2}	3.3×10^{-3}	8.5×10^{-4}
0.075	7.2×10^{-3}	1.5×10^{-2}	6.3×10^{-3}	6.2×10^{-3}	2.1×10^{-3}	1.7×10^{-3}
0.050	3.6×10^{-3}	6.3×10^{-3}	2.7×10^{-3}	2.7×10^{-3}	1.1×10^{-3}	1.4×10^{-3}

The first five columns are all the recovered stress, so they differ only by which boundary condition produced the field. The last two are the reaction and the multiplier on the same solves. Penalties at 10^4 , Nitsche at $\gamma = 10$, which are the values the leak table used.

Which treatment imposed the constraint stops mattering to the recovered stress. At cell 0.075 the rotated constraint gives 6.2×10^{-3} , the multiplier 6.3×10^{-3} , the penalty at 10^5 6.3×10^{-3} and Nitsche at $\gamma = 100$ is 6.6×10^{-3} . Those are the same number. What sets it is the recovery — a projection of a stress differentiated out of a piecewise-quadratic velocity — and not the boundary condition underneath. The reasoning this note began with — that a traction recovered from an approximate constraint inherits the approximation — is not what the measurement shows. It shows a floor that all of them share.

The reaction is about three times better than the recovery, on the same solve. 2.1×10^{-3} against 6.2×10^{-3} at cell 0.075, and it converges at the same rate rather than a better one. It costs nothing — it is the residual the solve has already assembled, and no field is differentiated to get it — and the timings below put a number on “nothing”.

The multiplier’s traction is the same story from the other side, and lands in the same place: 8.5×10^{-4} to 1.7×10^{-3} over the three finer meshes. Its column does not fall smoothly with h , because part of what it reports is the augmentation times the constraint residual, and how far a particular solve drove that residual is not a function of the mesh.

The parameter that was enough for the leak is not enough for the stress. Nitsche at $\gamma = 10$ leaks 1.2×10^{-3} in this problem and gets the stress amplitude 1.5% wrong. At $\gamma = 100$ the leak improves by a factor of nearly forty and the stress by a factor of two, to the recovery floor, where more γ buys nothing. Reading the leak alone would have said $\gamma = 10$ was converged.

4.d. The multiplier and the consistent boundary flux are the same object

The correction above is not a patch. Write the momentum row’s boundary term out and the identity is immediate: the assembled load is $M_\Gamma (\lambda + r(\mathbf{u} \cdot \hat{\mathbf{n}} - \tilde{u}_n))$, and at convergence it balances the volume residual restricted to the boundary, which is precisely the nodal load the consistent boundary flux back-calculation reads (Zhong et al., 1993). So

$$\lambda + r(\mathbf{u} \cdot \hat{\mathbf{n}} - \tilde{u}_n) = -M_\Gamma^{-1} \left(A \mathbf{u} - \mathbf{b} \right)_\Gamma,$$

\$\$

the CBF traction de-smearred with the boundary mass. The multiplier is not a second, independent estimate of the surface stress: it is the same computation the rotated constraint’s reaction performs, arrived at by carrying the traction as an unknown instead of reading it out of the residual afterwards. Dropping the r term is dropping part of the load, which is why it fails exactly where r is large.

Measured across the two solves — they are different discrete problems, so this is agreement rather than an identity check — the corrected multiplier and the rotated reaction differ by 3.2% at a contrast of 100 and 4.9% at 10^6 , both inside each route’s own error against the exact answer (5 to 9%). The identity also says why the two cannot be read off a single solve. On a multiplier-constrained boundary the constraint enters the row it constrains, so the assembled residual there is balanced at convergence and the back-calculation reads zero — measured, rms 4×10^{-13} against a traction of 0.37. There is no reaction left in the residual because the multiplier is holding it. The two routes are alternatives, not a cross-check available at the same time.

This also settles a question the free-surface work left open. That work used SolCx the same way, to choose among topography recoveries, and landed on a rotated free-slip lid with the CBF reaction and a continuous

pressure — corr 0.999, relative l_2 0.04 — while rejecting the multiplier. Both conclusions were right about what was in front of them: the multiplier *as returned* is missing the augmentation share, and the CBF reaction is the same quantity with nothing missing.

4.e. The other half: a lateral viscosity contrast

No exact solution has both a curved boundary and a laterally varying viscosity, so the case where weak constraints are most often reported to give trouble is a separate test with a trivial geometry. SolCx is that test: the unit box, free slip on all four walls, viscosity 1 to the left of $x = 0.5$ and η_B to the right. `uw.analytic.SolCx` publishes the exact dynamic topography on the top wall. Three walls carry the ordinary component condition and the treatment under test is on the top wall alone.

On a box every treatment reduces to holding one velocity component, so nothing here is about normals. What it can say is whether a treatment holds the traction it was given when the viscosity beside it jumps.

Relative l_2 error of the surface topography along the top wall, mean removed, at 32×32 elements. Each entry is the whole wall and then the wall with two elements trimmed from each end.

η_B/η_A	component Dirichlet	penalty, 10^4	multiplier	rotated
10	0.048 / 0.054	0.045 / 0.051	0.048 / 0.054	0.048 / 0.054
10^2	0.072 / 0.081	0.056 / 0.060	0.072 / 0.081	0.072 / 0.081
10^3	0.075 / 0.084	0.234 / 0.230	0.075 / 0.084	0.075 / 0.084
10^4	0.076 / 0.085	0.698 / 0.697	0.076 / 0.085	0.076 / 0.085
10^6	0.076 / 0.085	0.992 / 1.000	0.075 / 0.084	0.076 / 0.085

The three exact treatments agree to three figures at every contrast, whole wall and trimmed alike. That is the result to take from this half, and it took the multiplier reporting the whole traction rather than λ alone, and the rotated constraint holding the corner where it meets the side walls.

Read the first column as the floor. The component Dirichlet condition is exact and has no parameter, and its velocity error is 8.8×10^{-6} at a contrast of 10^6 . It still reads 0.085. That number is the recovery's error, not a boundary condition's: on the stiff half the recovered σ_{zz} is a difference between the pressure and $2\eta \partial_z u_z$ with $\eta = 10^6$, so a relative velocity error of 10^{-5} arrives in the stress at the size of the signal. Nothing here is worse than the reference except the penalty.

A bare penalty coefficient cannot serve both halves. At 10^4 it is the best column in the table at low contrast — the constraint is weak enough not to fight the recovery — and by 10^6 it is useless: 0.992, which is to say the recovered topography carries none of the signal. Scaling the coefficient by the local viscosity is the obviously right thing to want, and it does not solve here at any magnitude we tried, from μ to $10^3 \mu$ (a `Piecewise` viscosity inside the boundary term fails the line search). This is the same lesson as γ 's window on the annulus, in a place where the window closes entirely.

Nitsche is missing from this table, and honestly so. Our configuration of it on this box converges at a contrast of 10^6 and fails the line search at 10 — the opposite way round from every expectation — at 16×16 and 32×32 alike and at $\gamma = 10, 100$ and 1000. Where it does converge, γ has to rise with the contrast exactly as the annulus said: at 10^6 and 16×16 the surface stress error is 25 at $\gamma = 10$, 1.2 at 100 and 0.17 at 1000, while

the constraint is held to 10^{-3} or better throughout. We are not confident enough in that configuration to put a column of numbers behind it.

4.f. *What each one costs*

Seconds on the annulus, uniform viscosity, one core, direct solver: the solve, and then the surface traction by whatever route that treatment has. Median of three timed repeats after an untimed warm-up, run sequentially. Two sizes, because below about ten thousand nodes the four are separated by less than the run-to-run spread and there is nothing to read.

velocity nodes	penalty	Nitsche	multiplier	rotated
28 338	0.17 / 0.273	0.18 / 0.267	0.26 / —	0.16 / 0.010
71 424	0.45 / 0.631	0.47 / 0.657	0.67 / —	0.43 / 0.016

The dash is not a missing measurement. **The multiplier has nothing to recover:** λ is a finite element field in its own right, so its nodal values are the traction, pointwise, and $\lambda + r(\mathbf{u} \cdot \hat{\mathbf{n}} - \tilde{u}_n)$ is an expression evaluated where it is wanted. Reading it costs whatever reading a field costs — a few milliseconds of array indexing at this size, and nothing that belongs in a column beside a solve.

The rotated constraint's reaction does need one step, and it is worth being precise about which. (The 10 to 16 ms in the table is the reaction the solve already stashed; `boundary_flux` re-assembles the residual from scratch and costs 0.2 s, which is the 0.206 s in the comparison above.) The reaction is an *integrated* nodal load, $\int_{\Gamma} \sigma_{nn} \phi_i dS$, which is M_{Γ} times the pointwise traction. Turning it into a pointwise value means undoing that boundary mass. On a 2-D trace, and on 3-D P1 triangles, the lumped mass is diagonal and undoing it is a division — the 10 to 16 ms above. On **3-D P2 triangles it is a genuine solve**: the lumped row sums vanish at the vertices, so the consistent trace mass has to be assembled and solved, and Underworld gathers the trace to one rank to do it. That is the one place the CBF route pays for being a back-calculation, and it is the case a spherical free surface runs in.

So the conceptual difference the timings expose is not speed but *what each method hands you*: the multiplier gives the traction as a field, and the reaction gives it as a load that still has to be divided by a mass.

The multiplier's solve costs about 50% more, consistently — 0.67 s against 0.43 s at 71 000 nodes. That is the extra field and the larger saddle point. Rotating the degrees of freedom costs nothing measurable against the weak forms: the rotation is a sparse orthogonal transform on a boundary's worth of rows.

The weak forms have to recover theirs by differentiating the solution, and the projection that does it costs *more than the Stokes solve did* — 0.63 s against 0.45 s — so asking a penalty or Nitsche model for its surface stress roughly doubles the timestep.

The obvious question is whether that is the method's cost or the recovery's. A global L^2 projection to get values on a thousand boundary nodes is plainly more work than the job requires, and the consistent boundary flux is right there, reading the assembled residual rather than differentiating anything. **It does not work for a weakly imposed condition**, and the reason is the same one that makes it unavailable to the multiplier:

cell 0.0125	projection	CBF back-calculation
penalty, node normal	0.651 s, error 1.1×10^{-3}	0.224 s, error 1.00

cell 0.0125	projection	CBF back-calculation
Nitsche	0.666 s, error 3.6×10^{-4}	0.230 s, error 1.00
rotated	0.602 s, error 1.6×10^{-4}	0.206 s, error 1.6×10^{-4}

An error of 1.00 is the metric reporting that nothing was recovered. A reaction exists in the residual only where a row has been *constrained*; a weak condition supplies its traction as a term inside the row it acts on, so the residual there is balanced at convergence and there is nothing left to read. The multiplier does the same thing, and gets away with it because the term it supplies, λ , is the traction as a field. Nitsche's term is written in terms of $\sigma(\mathbf{u})$, so reading it back still means differentiating the answer.

That is the structural statement the timings are really making, and it follows the two pairs exactly:

how the constraint is imposed	the traction is	to read it
weakly, by a term (penalty, Nitsche)	a by-product	differentiate the solution
exactly, by construction (rotated)	the constraint reaction	de-smear the nodal load
exactly, by a multiplier	an unknown of the system	read the field

The cost of the first row is negotiable — a recovery restricted to the boundary would be cheaper than a global projection — but the differentiation is not.

What these timings are not

Two-dimensional, one core, direct solver. What they measure is the difference between a recovery *solve* and boundary arithmetic, which is structural and survives scaling. They say nothing about a large parallel spherical shell, where the rotated velocity block's multigrid and the multiplier's larger Schur complement are the terms that matter and neither is exercised here.

4.g. Which one to use

For a model that consumes the velocity and nothing else, all four are the same to plotting accuracy — provided the constraint is written against the node normal. That proviso is the only one that can spoil the velocity, and it costs one line.

When the wall-normal traction is the answer:

- **Rotated free slip is the default.** It holds the constraint to machine precision rather than to the discretisation, its reaction is the most accurate surface stress measured here, and that reaction is nearly free. The price is structural — a mixed basis that the multigrid has to carry — and it is paid once, inside the solver, rather than by the person setting up the model.
- **The multiplier is its equal on accuracy** and returns the same object by a different route; take it when you want the traction as an unknown of the system, or when the constrained problem's conditioning suits you better. It costs about 50% more to solve.
- **Nitsche is the one to reach for when the boundary condition must change during the model** — a wall that begins as a prescribed velocity and relaxes to a prescribed traction is a Nitsche problem, because a hard constraint cannot morph. Budget for tuning γ against the stress and not against the leak, and expect the window to move with the viscosity contrast.

- **A direct penalty is fine for a velocity-only model** and needs the node normal, a coefficient chosen per problem, and a check on something physical before the answer is believed.

4.h. Which is to say, a free surface

This note began with a free surface, and that is where the argument lands. A free surface needs σ_{nn} every step: the surface moves under the traction it carries, so the recovery is not something done once at the end of a run but part of the timestep, alongside the Stokes solve it follows.

Everything above then reads differently. The projection that costs more than the Stokes solve is paid at every step, and doubles the model. The differentiation it performs is applied to a velocity field that a weak condition never made satisfy $\mathbf{u} \cdot \hat{\mathbf{n}} = 0$ exactly, on a boundary that is moving. And the surface stress recovered that way has failure modes of its own that have nothing to do with the boundary condition — a continuous pressure checkerboards at a viscosity jump, a discontinuous one puts a node-to-node zigzag into the reaction on a simplex boundary — so the recovery has to be tuned against the element types as well.

The two exact treatments hand the traction over as part of the answer, which is the property a free surface wants. Between them:

- **the rotated constraint** gives the reaction, and on the 3-D P2 triangular trace a deforming spherical surface actually uses, de-smearing it is a consistent-mass solve that Underworld gathers to one rank. That is a real serial step inside a parallel timestep;
- **the multiplier** gives a field, so there is nothing to de-smear at all — in 2-D or in 3-D. It costs about 50% more in the solve.

We have not measured that trade on a large parallel shell, and it is the measurement worth doing next: the multiplier's premium is a fixed fraction of a solve, while the gather is a serial section, and which one wins is a question of rank count rather than of method.

5. USING IT

```
import underworld3 as uw

mesh = uw.meshing.Annulus(radiusInner=0.5, radiusOuter=1.0, cellSize=0.05)
stokes = uw.systems.Stokes(mesh)

# Value first: 0 is free slip. A non-zero scalar or expression prescribes the
# wall-normal datum  $\mathbf{u} \cdot \mathbf{n} = u_n$  strongly instead.
stokes.add_rotated_freeslip_bc(0.0, "Upper")
stokes.add_rotated_freeslip_bc(0.0, "Lower")

stokes.solve()

# The constraint reaction, which is the boundary normal traction.
sigma_nn = stokes.boundary_normal_traction("Upper")
```

Leave the normal to Underworld unless the constraint has to follow the true surface rather than the mesh. Passing an analytic normal $-\mathbf{x} / |\mathbf{x}|$ on a sphere — is exact for the geometry and keeps a consistency error against the faceted assembly, which is usually not what you want.

Reach for Nitsche when the boundary condition has to **change during the model**. A hard constraint cannot morph: a wall that begins as a prescribed velocity and relaxes to a prescribed traction is a Nitsche problem, because the rotated constraint is either imposed or it is not.

REFERENCES

- Behr, M. (2004). On the application of slip boundary condition on curved boundaries. *International Journal for Numerical Methods in Fluids*, 45(1), 43–51. <https://doi.org/10.1002/fld.663>
- Engelman, M. S., Sani, R. L., & Gresho, P. M. (1982). The implementation of normal and/or tangential boundary conditions in finite element codes for incompressible fluid flow. *International Journal for Numerical Methods in Fluids*, 2(3), 225–238. <https://doi.org/10.1002/fld.1650020302>
- Kramer, S. C., Davies, D. R., & Wilson, C. R. (2021). Analytical solutions for mantle flow in cylindrical and spherical shells. *Geoscientific Model Development*, 14(4), 1899–1919. <https://doi.org/10.5194/gmd-14-1899-2021>
- Nitsche, J. (1971). "Über ein Variationsprinzip zur Lösung von Dirichlet-Problemen bei Verwendung von Teilräumen, die keinen Randbedingungen unterworfen sind. *Abhandlungen Aus Dem Mathematischen Seminar Der Universität Hamburg*, 36(1), 9–15. <https://doi.org/10.1007/bf02995904>
- Zhong, S., Gurnis, M., & Hulbert, G. (1993). Accurate determination of surface normal stress in viscous flow from a consistent boundary flux method. *Physics of the Earth and Planetary Interiors*, 78(1–2), 1–8. [https://doi.org/10.1016/0031-9201\(93\)90078-n](https://doi.org/10.1016/0031-9201(93)90078-n)